

Desktop BLE and Serial Communication

Connection, reading, writing, INFO, SIP, and response handling

Architecture rule

Native communication stays in the Electron main process. React uses only the typed preload API. Keep contextIsolation and sandbox enabled, keep nodeIntegration disabled, and validate every renderer-to-main IPC request.

Communication path

```
React renderer -> preload API -> validated IPC -> deviceManager
                  -> bleService OR serialService -> target

target -> BLE notification / serial data -> MessageFramer
                  -> deviceManager event -> preload subscription -> React
```

Core prerequisites

Area	Required project contract
Runtime	Electron 44, Electron Forge 7, TypeScript; install with <code>npm install</code>
BLE	@abandonware/noble; powered adapter; OS permission; target service UUID
Serial	serialport + native binding; connected port; correct baud rate (app default 115200)
Native build	Keep Noble and SerialPort external in Vite; Forge rebuilds ABI; AutoUnpackNativesPlugin stays enabled
macOS package	Keep NSBluetoothAlwaysUsageDescription and NSBluetoothPeripheralUsageDescription

Transport independence

BLE and serial may be connected simultaneously. Each keeps separate connection, terminal, SIP mode, and Settings state. Only one device is allowed per transport.

Start: `npm start` | Validate: `npm run typecheck`, `npm run lint`

1 Connection process

Register renderer subscriptions before starting discovery or connecting, then let the main process own the native lifecycle.

BLE renderer API

```
const off = window.electronAPI.ble.onDeviceDiscovered(onDevice);
await window.electronAPI.ble.startScan();
await window.electronAPI.ble.connect(selectedDeviceId);
// React cleanup: off();
```

BLE main-process sequence

1	Wait for Noble state poweredOn; reject unauthorized or unsupported.
2	Scan only for service 0bd51666-e7cb-469b-8e4d-2742f1ba77cc (15-second timeout).
3	Publish each discovered id, name, and RSSI to the renderer.
4	Stop scanning and disconnect any previous BLE peripheral.
5	Connect, then discover Command and Settings characteristics.
6	Require Command; keep Settings when found.
7	Subscribe to characteristics that advertise notify or indicate.
8	Register BLE in deviceManager and publish connected status.

Serial main-process sequence

1	Call SerialPort.list() and return available port metadata.
2	Renderer selects a port path and baud rate.
3	Close any previously open serial port.
4	Reset message framer, idle timer, and in-memory settings.
5	Open new SerialPort({ path, baudRate }).
6	Attach data, close, and error handlers.
7	Register serial in deviceManager and publish connected status.

Serial renderer API

```
const ports = await window.electronAPI.serial.listPorts();
await window.electronAPI.serial.connect(ports[0].path, 115200);
```

BLE UUID	Purpose
e7add780-b042-4876-aae1-11285535f821	Command characteristic
e7add780-b042-4876-aae1-11285535f721	Settings characteristic

2 Reading BLE and serial data

Both transports use MessageFramer and publish the same DeviceMessage shape. The byte-delivery mechanisms differ.

BLE read

```
characteristic.on('data', (chunk: Buffer) => {
  for (const line of framer.push(chunk)) {
    publishMessage({ transport: 'ble', source, direction: 'in',
      text: line.toString('utf8'), hex: line.toString('hex') });
    parseAndPublishFuk(line);
  }
});
await characteristic.subscribeAsync();
```

- Use one framer per characteristic so Command and Settings fragments never mix.
- Do not assume one notification equals one message.
- Split on CR or LF. The target may send CR and delay the standalone LF.
- FUK normally arrives on Command, but parse FUK from either subscribed characteristic.

Serial read

```
port.on('data', (chunk: Buffer) => {
  for (const line of framer.push(chunk)) handleIncomingLine(line);
  resetIdleTimer(200, () => {
    const finalLine = framer.flush();
    if (finalLine) handleIncomingLine(finalLine);
  });
});
```

- Serial is one continuous stream: a data event can contain a fragment, one line, or several lines.
- Split on CR or LF, exactly like BLE.
- Reset the 200 ms idle timer after every chunk; flush only the remaining unterminated bytes.

Read concern	BLE	Serial
Native event	Characteristic data notification	Port data event
Framer instances	One per characteristic/source	One per open port
Line boundary	CR or LF	CR or LF; idle flush fallback
Published source	command or settings	serial
Renderer API	device.onMessage	device.onMessage

Every-FUK state rule

After publishing the raw line to the correct terminal, parse FUK. A valid FUK replaces that transport's main-process settings, publishes device:settings-changed, updates only the matching React settings state, and also resolves a pending settings write.

3 Writing BLE and serial data

The renderer always names the transport. The main process selects the active BLE or serial implementation.

Renderer API

```
const bytes = new TextEncoder().encode('RAW_COMMAND');
await window.electronAPI.device.writeCommand('ble', bytes);
await window.electronAPI.device.writeCommand('serial', bytes);

await window.electronAPI.device.writeSettings('ble', { brightness: 4 });
await window.electronAPI.device.writeSettings('serial', { brightness: 4 });
```

Write	BLE	Serial
Raw command	Command characteristic; exact bytes; write with response; no terminator added	Open port; exact bytes; no terminator added
INFO / SIP	Settings characteristic; plain ASCII; no terminator	Open port; CRLF appended by deviceManager
Settings	Settings characteristic; FU1 then FU2; no terminator	One FUK-shaped line plus CRLF; drain port

Settings wire formats

```
BLE (two GATT writes, no terminator)
FU1:01:0<brightness>:00:<mode>:0<shootingArea>
FU2:0<shotsHeat>:<secondsHeat>:<HH:mm>

Serial (one line, then CRLF)
FUK:01:0<brightness>:00:<mode>:0<shootingArea>:0<shotsHeat>:<secondsHeat>:0000000\r\n
```

Field	Allowed
Brightness	1-5
Mode	0 Competition; 1 Training; 2 OCR
Shooting area	0-2
Shots per heat	1-5
Heat seconds	10, 20, 30, 40, 50

Settings confirmation sequence

Step	Behavior
1	Merge partial update into that transport's latest FUK-derived state.
2	Skip the write if no value changed.
3	Send the transport-specific format.
4	Wait up to 10 seconds for the next valid FUK on the same transport.
5	Resolve with device-reported values; reject timeout or disconnect.

Standby only

Expose settings controls only after the selected transport confirms SIP mode S. Do not apply only the requested value; the incoming FUK is authoritative.

4 INFO and SIP write path

INFO and SIP use identical command text on BLE and serial. Only destination and terminator differ.

End-to-end send path

```
React method
-> preload ipcRenderer.invoke
-> trusted ipcMain handler
-> deviceManager.writeInfo / writeSip
-> encodeInfo / encodeSip
-> deviceManager.writeLine
-> active transport.writeGenericCommand
```

Formats

```
INFO01
SIP:01:<S|L>:hhmmsscc
Example: SIP:01:L:08263283
```

- Lane/target is currently fixed to 01.
- S or L is the requested SIP sync kind.
- hhmmsscc is local hour, minute, second, and centisecond: eight zero-padded digits.

Renderer calls

```
await window.electronAPI.device.writeInfo('ble');
await window.electronAPI.device.writeInfo('serial');
await window.electronAPI.device.writeSip('ble', 'S');
await window.electronAPI.device.writeSip('serial', 'L');
```

Command	BLE wire	Serial wire
INFO	Settings characteristic: INFO01; no terminator	Open port: INFO01\r\n
SIP	Settings characteristic: encoded SIP; no terminator	Open port: encoded SIP plus CRLF

Critical BLE routing

Send both INFO and SIP to the BLE Settings characteristic, not the Command characteristic. Serial has one wire, so no characteristic routing exists.

Asynchronous contract

INFO and SIP are fire-and-forget sends. Their promises resolve after bytes are written and the outgoing terminal entry is published. They do not wait for a device response.

5 INFO and SIP response handling

Responses return through the normal BLE-notification or serial-data read pipeline and are delivered asynchronously to the renderer.

SIP response

```
Request: SIP:01:L:08263283
Response: SIP:01:L:08263283:11714
```

- BLE: receive through a subscribed characteristic notification.
- Serial: receive through port.on('data').
- Both: frame on CR/LF and publish to the terminal.
- Renderer regex `^SIP:\d+:([SL])`: extracts the confirmed kind.
- Confirmed S enables Standby settings; L represents Live mode.
- The extra final response field is device-defined and currently not interpreted.

INFO response

```
Request: INFO01
Replies: IN0:95:88:91:70
        IN1:100:93:-1:84
```

- `IN<rowIndex>:<value1>:...` carries one discovery row; documented indexes are 0-9.
- `parseMasterDiscoveryRow` converts numeric fields and normalizes empty/invalid values to -1.
- The current UI highlights `IN<number>` replies but does not store a discovery table.

Implementation checklist

Transport boundary	Read/state boundary
1. Keep Noble/SerialPort in main.	6. Append CRLF to serial protocol lines.
2. Keep native modules external and unpacked.	7. Frame incoming data on CR or LF.
3. Route BLE raw commands to Command.	8. Apply every FUK only to its transport.
4. Route BLE settings, INFO, SIP to Settings.	9. Treat INFO/SIP replies asynchronously.
5. Never add serial endings to BLE.	10. Clean up renderer subscriptions.

Primary code map

File	Purpose
<code>bleService.ts</code>	BLE scan, connect, notifications, writes
<code>serialService.ts</code>	Port list, connect, stream reads, writes
<code>deviceManager.ts</code>	Transport selection, INFO/SIP routing, broadcasts
<code>messageFramer.ts</code>	CR/LF framing and serial idle flush support
<code>deviceCommandProtocol.ts</code>	INFO/SIP encoders and IN row parser
<code>deviceSettingsProtocol.ts</code>	FU1/FU2/FUK encoders and FUK parser
<code>preload.ts</code> / <code>useDevice.ts</code>	Typed renderer API, subscriptions, UI state